

多房并发投屏会议软件

1. 需求概述

项	说明
业务场景	方案设计公司将本地 PC 上的方案视频资料投屏到会议房间，客户在手机端观看并进行语音/视频讨论；公司方不参与讨论
房间模型	多房间并发；单房间内只有 2 个客户手机端，房间内没有公司 PC 端。 公司 PC 端以后台身份运行，不出现在房间成员中（仅作为隐藏推流源与管理端）
资料安全	方案资料 只存储在本地 PC ，不上传云端；仅做 实时流传输 ，服务器不存储任何媒体内容
入会方式	公司在 PC 端创建房间 → 生成二维码 → 发给客户 → 客户手机 App 扫码自动进入房间
分发方式	Android APK 私发安装，不上架应用商店（iOS 可选 TestFlight/企业签名，见 § 9）
会议能力	类腾讯会议：语音通话、视频通话、屏幕/窗口/视频文件投屏、静音、摄像头开关等； 其中手机端"视频通话/摄像头"功能是否开放，由 PC 端按房间设置开关控制

1.1 补充需求（沟通确认，2026-08-14）

- 房间模型修正：**单个房间只有 2 个手机客户端成员；公司 PC 端不作为房间成员出现，以后台身份推流和管理。
- 会议功能开关：**手机端"视频通话"和"摄像头"功能，需要在 PC 端提供设置选项，可按房间选择开放或关闭。
- 多房间不同内容并发投放：**PC 端可在不同时间选择不同内容投给不同房间。例如：8:00 选择内容 1 投给 A 房间，8:01 选择内容 2 投给 B 房间；两个房间并行运行，**不串音、不串频**。
- 房间就位与播放控制：**公司开好房间、投入视频内容后等待；当 2 个手机客户端全部就位后，**触发 PC 端显示"该房间已运行"**；手机客户端可以操作：**开始播放 / 暂停 / 拖拉进度条 / 调节明暗 / 调节声音**。
- 会议时间：**房间需显示会议时间；PC 端可设置会议时长，**时间到后房间自动关闭**。
- 点赞：**房间内（手机端）有点赞按钮，点赞事件实时连接到 PC 端展示，并在后端做记录。

7. **截屏/录制策略修正**：房间内手机端**允许截屏，但禁止录制（录屏）**。（原方案的 FLAG_SECURE 会同时禁止截屏与录屏，不再采用，改为录屏检测 + 水印方案，见 § 6）

2. 总体架构

要点：

- 媒体流走 SFU（选择性转发单元）转发，服务器只做实时 RTP 包转发，**不解码、不录制、不存储**，满足"资料不上云"的合规要求。
- 1 房间固定 2 个手机媒体端（上下行音视频）+ 1 路 PC 隐藏推流（只发不收、不显示为成员），SFU 模式带宽和性能远优于 MCU，且无云端合流留存风险。
- **每个房间对应一个独立的 SFU Room 实例**，PC 端为每个房间建立独立的 RTC 连接与独立播放器实例，媒体轨道按房间隔离，天然保证多房并发时不串音、不串频。
- 业务面（房间/二维码/鉴权/点赞记录/会议时长）与媒体面（信令/SFU）分离，便于独立扩容。

3. 技术选型

层	选型	说明
手机 App	Flutter 3.x + flutter_webrtc	一套代码 Android/iOS；mobile_scanner 扫码；permission_handler 权限
PC 投屏端	Flutter Desktop (Windows) + flutter_webrtc	getDisplayMedia 屏幕/窗口捕获；qr_flutter 生成二维码；本地视频文件经 media_kit 解码后推流； 多房间时每房间独立播放器 + 独立 RTC 连接
后端业务	Java 17 + Spring Boot 3	REST API + Spring Security (JWT)
信令	Spring WebSocket (STOMP 或原生 WS)	房间信令、成员事件、SDP/ICE 交换、 播放控制指令、点赞事件、会议倒计时
SFU 媒体服务器	LiveKit（推荐，自部署） 或 mediasoup / Janus	LiveKit 自带 SFU + 信令 + 各端 SDK（含 Flutter SDK livekit_client），自部署在自己服务器上，关闭录制功能即"零留存"；Java 侧用 LiveKit Server API 生成入会 Token
NAT 穿透	coturn (STUN/TURN)	自部署，保障复杂网络下连通

数据库	MySQL 8 / PostgreSQL	仅存房间、用户、会议记录、点赞记录等元数据， 不存媒体
部署	Docker Compose (Nginx + Spring Boot + LiveKit + coturn + MySQL)	单机即可支撑数十房间并发

备选：若坚持完全自研信令 + 原生 WebRTC，可去掉 LiveKit，由 Spring WebSocket 自己做 SDP/ICE 转发 + mediasoup 做 SFU；但 LiveKit 方案开发量小、稳定性高、Flutter SDK 成熟，**强烈推荐**。

4. 核心流程

4.1 创建房间与扫码入会

1. PC 端登录（公司账号）→ 调 `POST /api/rooms` 创建房间（可同时设置：**会议时长、是否开放视频通话/摄像头**）。
2. 后端生成 `roomId` + 一次性入会凭证 `inviteToken`（带过期时间、房间号、最大 2 人限制），返回入会链接：`meeting:///join?room=xxx&token=yyy`（自定义 scheme）。
3. PC 端将链接渲染为二维码，微信/其他渠道截图发给客户。
4. 客户打开 App → 扫码 → App 解析 token → 调 `POST /api/rooms/join` 校验 → 后端签发 LiveKit 入会 JWT → App 连接 SFU 入会。
5. 房间人数控制：后端校验客户端数量 ≤ 2 （**PC 投屏端为后台隐藏角色，不计入成员、不出现在成员列表**），超员拒绝。

4.2 PC 投屏（核心）

- **屏幕/窗口投屏**：`flutter_webrtc` `getDisplayMedia` 捕获整屏或指定窗口 → 作为视频轨推流到 SFU。
- **本地视频文件投放（推荐）**：PC 端用 `media_kit` 解码本地文件，逐帧喂给 WebRTC 视频轨（画质更稳，支持暂停/进度控制，可响应手机端的播放控制指令）。
- **多房并发投放**：PC 端维护"房间-内容"绑定列表，可在不同时间为不同房间选择不同内容；每房间独立的播放器实例 + 独立 RTC 连接 + 独立音频轨，互不干扰（不串音、不串频）。
- **投屏参数**：1080p/30fps，H.264（硬编），码率 2–4 Mbps 自适应（WebRTC 自带拥塞控制，弱网自动降级）。
- **音频**：投放视频的伴音走播放器解码音轨直接推流（文件投放场景）；整屏/窗口投屏场景走系统回环采集（WASAPI loopback），保证客户能听到视频声音。

4.3 客户端会议（类腾讯会议）

- 主画面：PC 投屏流（大窗）；小窗：对方客户视频（可拖动/隐藏；仅当 PC 端开放摄像头/视频通话时显示）。
- 功能：麦克风静音/取消、摄像头开/关（受 PC 端房间设置控制）、前后摄像头切换、扬声器/听筒切换、离开会议、网络质量指示、**点赞按钮**、**会议时间/剩余时长显示**。
- **播放控制**：房间运行中，手机端可对投放的视频内容执行 开始播放/暂停/拖动进度条；**明暗、音量为手机本地调节**（不影响另一客户端）。
- 双客户互相语音/视频讨论；PC 端只推流不订阅客户音视频（公司不参与讨论），也可选择性监听。

4.4 房间就位与运行状态

1. 公司在 PC 端开好房间，选好投放内容，处于"等待就位"状态。
2. 两个手机客户端全部入会后，后端判定"就位"，推送事件给 PC 端，**PC 端显示"该房间已运行"**，同时开始会议计时。
3. 运行中：手机端播放控制指令经信令通道转发给 PC 端播放器执行，播放状态（播放中/暂停/进度）向房间内广播同步。

4.5 会议结束

- 手动结束：PC 端点"结束会议"→ 后端销毁房间 → SFU 踢出全部成员 → invite token 失效。
- **自动结束：会议时长到期，后端定时任务自动关闭房间**（同上流程），到期前向房间内推送倒计时提醒（如剩余 5 分钟/1 分钟）。
- 服务器仅留会议元数据日志（时间、房号、时长、**点赞记录**），无任何媒体内容。

5. 后端接口设计（示意）

POST /api/auth/login	公司账号登录（PC 端）
POST /api/rooms	创建房间（含会议时长、功能开关）→ {roomId, inviteUrl, qrContent, expireAt}
POST /api/rooms/join	客户扫码入会 {roomId, inviteToken} → {livekitToken, wsUrl}
POST /api/rooms/{id}/close	结束会议
GET /api/rooms/{id}/members	房间成员列表（仅手机客户端）
PUT /api/rooms/{id}/settings	修改房间功能开关（视频通话/摄像头 开/关）
POST /api/rooms/{id}/like	点赞（手机端）
GET /api/rooms/{id}/likes	点赞记录/统计（PC 端）
WS /ws/rooms/{id}	房间事件（成员进出、就位、播放控制、点赞、 倒计时、房间关闭）

数据表（元数据）：`user`（公司账号）、`room`（房号/状态/创建人/会议时长/过期时间/功能开关）、`room_member`（入会记录）、`invite_token`、`room_like`（点赞记录）。

6. 安全设计

项	措施
传输加密	全链路 HTTPS/WSS；媒体流 WebRTC 强制 DTLS-SRTP 加密
资料不落地	SFU 关闭录制/转推，服务器无任何媒体写盘逻辑；可在合同/审计层面出具"零留存"说明
入会控制	一次性 inviteToken（短时效 + 绑定房间 + 限用次数）；房间人数硬限制 2 客户端
截屏/防录制	允许截屏、禁止录屏 ：不使用 FLAG_SECURE（其会连截屏一起禁掉）。Android：录屏检测（Android 15+ ScreenRecordingCallback；低版本检测 MediaProjection/录屏服务）→ 检测到录屏时遮挡画面并上报；iOS：UIScreen.isCaptured 监听，录屏时遮挡画面。辅以 App 端加水印（显示客户手机号/时间）威慑翻拍
鉴权	公司账号 JWT；客户免注册，凭 token 匿名入会（降低使用门槛）

7. 并发与部署规模

- 单房间带宽：PC 上行 ~4 Mbps，SFU 下行 4 Mbps×2（投屏）+ 客户互视频 ~1 Mbps×2 ≈ 约 15 Mbps/房。
- 一台 4C8G 云主机 + 100 Mbps 带宽 ≈ 6-8 房并发；带宽是主要瓶颈，按房间数线性扩带宽即可。
- PC 端多房并发受限于本机编码能力：1080p/30fps 硬编下，普通商用机建议同时投放 ≤ 3-4 房，超出可降分辨率或加 PC。
- LiveKit 支持多节点水平扩展，后续房间量大时加节点即可，无需改代码。

8. 项目结构与里程碑

meeting-app/	Flutter 手机 App (Android/iOS)
meeting-desktop/	Flutter Windows 投屏端
meeting-server/	Spring Boot 业务 + 信令
deploy/	docker-compose (nginx/livekit/coturn/mysql)

9. APK 分发与 iOS 说明

- **Android:** flutter build apk --release 自签名，私发安装（引导用户开启"未知来源"）；App 内置版本检查接口，提示下载新 APK（下载页由后端提供）。
- **iOS**（如需要）：不上架情况下可用 TestFlight（最简单，100 外部测试员起步）或企业开发者证书签名分发；若客户仅安卓可先不做 iOS。

10. 风险与对策

风险	对策
客户网络差导致投屏卡顿	WebRTC 自适应码率 + simulcast 多档投屏分辨率；提示客户用 Wi-Fi
Windows 系统声音采集	使用 WASAPI loopback (flutter_webrtc 桌面端支持系统音频捕获；不支持时用虚拟声卡方案兜底)
客户录屏泄露资料	录屏检测 (Android 15+/iOS 系统 API) + 遮挡画面 + 水印 + 合同约定（技术无法 100% 防翻拍，需说明；低版本 Android 录屏检测能力有限，为尽力而为方案）
PC 多房并发编码压力	限制单机并发投放房间数；必要时降低投放分辨率/帧率或增加投屏 PC

手机端播放控制冲突 (两客户端同时操作)	控制指令带序号/时间戳，后端串行转发，PC 端按最后指令执行并广播权威状态
-------------------------	---------------------------------------